

Hora de tentar sozinhos, agora com Árvore de Decisão

Mesmo dataset mpg, mesma categoria economico que vocês já usaram nas práticas anteriores — agora aplicando Árvore de Decisão.

Diferente do KNN, aqui não precisa normalizar — árvores não medem distância, então a escala das features não importa.

Escolham 2 ou mais features — árvore lida bem com várias, sem precisar de tratamento especial.

Importações necessárias

Note que NÃO precisamos do StandardScaler dessa vez.

LEMBRETE DE SINTAXE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score
```



Categoria e features

Recriem economico, olhem a correlação, e escolham 2 ou mais features.

LEMBRETE DE SINTAXE

```
df = pd.read_csv("mpg_desafio_alunos.csv")
df["economico"] = (df["mpg"] >= 23).astype(int)

df.corr(numeric_only=True)["economico"]

X = df[["weight", "cylinders"]] # escolham as de vocês
y = df["economico"]
```



Tratar valores nulos

Necessário só se horsepower estiver entre as escolhidas.

LEMBRETE DE SINTAXE

```
df = df.dropna(subset=["horsepower"])
```



Separar treino e teste

stratify=y de novo, exatamente como nas práticas anteriores.

LEMBRETE DE SINTAXE

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=5, stratify=y  
)
```



Treinar a árvore

Comecem com `max_depth=3`, só pra ter um primeiro modelo funcionando.

LEMBRETE DE SINTAXE

```
modelo = DecisionTreeClassifier(max_depth=3, random_state=42)
modelo.fit(X_train, y_train)
```



Visualizar a árvore

Vejam o desenho das perguntas que o modelo aprendeu, com as features que vocês escolheram.

LEMBRETE DE SINTAXE

```
plt.figure(figsize=(14,7))
plot_tree(modelo, feature_names=X.columns.tolist(),
          class_names=["nao economico", "economico"],
          filled=True, rounded=True)
plt.show()
```



As perguntas mudaram de cara

Na árvore dos animais, as perguntas eram sempre 'sim ou não' (bota_ovos? voa?). Aqui, com weight/cylinders (números contínuos), a árvore pergunta coisas como 'weight $\leq$ 2800?'.

É o mesmo mecanismo por trás — a árvore sempre faz perguntas de sim/não. A diferença é que, com número contínuo, ela primeiro precisa escolher QUAL valor usar como corte.

Se aparecer um número estranho tipo 'weight $\leq$ 2289.50' na sua árvore, é exatamente isso — não é erro.

Avaliar com acurácia

Um primeiro número, sem aprofundar ainda.

LEMBRETE DE SINTAXE

```
pred = modelo.predict(X_test)
print(f"Acuracia: {accuracy_score(y_test, pred):.2%}")
```



Testar profundidades

De 1 a 10, achem qual max_depth dá a melhor acurácia com as features escolhidas.

LEMBRETE DE SINTAXE

```
profundidades = range(1, 11)
acuracias = []
for d in profundidades:
    m = DecisionTreeClassifier(max_depth=d, random_state=42).fit(X_train, y_train)
    acuracias.append(accuracy_score(y_test, m.predict(X_test)))

melhor_d = list(profundidades)[np.argmax(acuracias)]
print(f"Melhor max_depth: {melhor_d}")

plt.plot(list(profundidades), acuracias, marker="o")
plt.axvline(melhor_d, color="red", linestyle="--")
plt.show()
```



Ver importância das features

Quais das suas features escolhidas mais pesaram nas decisões?

LEMBRETE DE SINTAXE

```
importancias = pd.Series(  
    modelo.feature_importances_, index=X.columns  
)  
.sort_values()  
  
importancias.plot(kind="barh")  
plt.show()
```

Avaliação final

Retreinem com o melhor max_depth, e rodem no mpg_desafio_final.csv.

LEMBRETE DE SINTAXE

```
modelo_final = DecisionTreeClassifier(max_depth=melhor_d, random_state=42)
modelo_final.fit(X_train, y_train)

df_final = pd.read_csv("mpg_desafio_final.csv")
df_final["economico"] = (df_final["mpg"] >= 23).astype(int)

X_final = df_final[["weight", "cylinders"]]
y_final = df_final["economico"]

pred_final = modelo_final.predict(X_final)
print(f"Acuracia final: {accuracy_score(y_final, pred_final):.2%}")
```



Refletir

Comparando os 3 algoritmos que vocês já praticaram (Regressão Logística, KNN, Árvore): qual foi mais fácil de configurar? Qual deu resultado mais fácil de explicar pra alguém que não sabe ML?

Árvore é o único que você consegue 'desenhar' e explicar pra qualquer pessoa, sem matemática.

3 algoritmos praticados, mesmo dataset

Features escolhidas com dado, árvore visualizada, profundidade otimizada, avaliada no conjunto nunca visto — tudo com decisões de vocês.

A seguir: Random Forest — várias árvores, votando juntas.